



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

SMART LOGISTICS & TRANSPORTATION MANAGEMENT SYSTEM (SLTMS)

Case Study: Impano Gateway Logistics Ltd



Prepared By: Hirwa Roy (Student ID: 24174)

Course: Best Programming Practices and Design Patterns

Instructor: RUTARINDWA Jean Pierre

Academic Year: 2025 – 2026

SMART LOGISTICS & TRANSPORTATION MANAGEMENT SYSTEM (SLTMS)

Case Study: Impano Gateway Logistics Ltd

1. Introduction

The Smart Logistics & Transportation Management System (SLTMS) was developed to solve operational challenges faced by Impano Gateway Logistics Ltd.

The company handles cargo transportation, shipment coordination, driver assignment, warehouse operations, and customer deliveries across Rwanda and East Africa.

The current manual system based on phone calls and spreadsheets creates delays, billing errors, poor communication, and inefficient vehicle management.

The SLTMS digitizes all logistics operations and improves automation, tracking, and reporting.

2. Company Background

Impano Gateway Logistics Ltd is a logistics and transportation company that provides freight forwarding and cargo delivery services.

The company manages trucks, vans, and motorcycles for transporting cargo to different regions.

The company serves individual customers, businesses, warehouses, and retail suppliers.

Due to business growth, the company requires a digital system capable of handling large shipment volumes efficiently.

3. Problem Statement

Several operational problems were identified in the company:

- Manual shipment tracking causes delays and missing cargo information.
- Drivers are assigned manually, causing poor fleet utilization.
- Vehicle maintenance schedules are not monitored properly.
- Paper invoices create calculation and payment errors.

- Customers cannot track delivery progress in real time.
- Management struggles to generate operational reports.

These problems reduce efficiency and negatively affect customer satisfaction.

4. Proposed Solution

The proposed SLTMS automates logistics operations through a Java-based application.

Main features include:

- Client management
- Driver registration
- Fleet management
- Shipment booking
- Driver and vehicle assignment
- Invoice generation
- Shipping cost calculation
- Report generation

The solution improves operational speed, accuracy, and reliability.

5. System Objectives

General Objective:

To design and implement a logistics management system that automates transportation operations.

Specific Objectives:

- Reduce paperwork and manual processing
- Improve shipment tracking
- Automate invoice generation
- Improve customer service
- Improve reporting and business decisions
- Apply object-oriented software engineering principles

6. Functional Requirements

The SLTMS contains several modules:

1. Client Management

Registers and manages customer information.

2. Driver Management

Stores driver details and availability status.

3. Fleet Management

Manages vehicles including trucks, vans, and motorcycles.

4. Shipment Booking

Creates shipments and assigns available resources.

5. Invoice Management

Generates invoices automatically and tracks payment.

6. Report Management

Generates shipment and revenue reports.

7. Non-Functional Requirements

The system also satisfies non-functional requirements:

- Reliability – accurate shipment and invoice management
- Maintainability – modular architecture for easy updates
- Scalability – supports future expansion
- Performance – fast shipment processing
- Portability – runs on multiple operating systems
- Usability – easy console interface

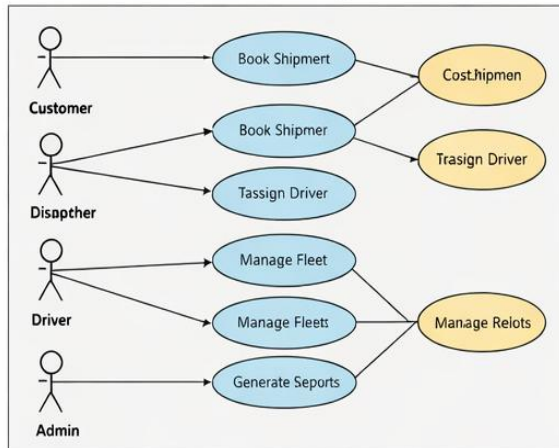
8. UML Diagrams

The project includes several UML diagrams:

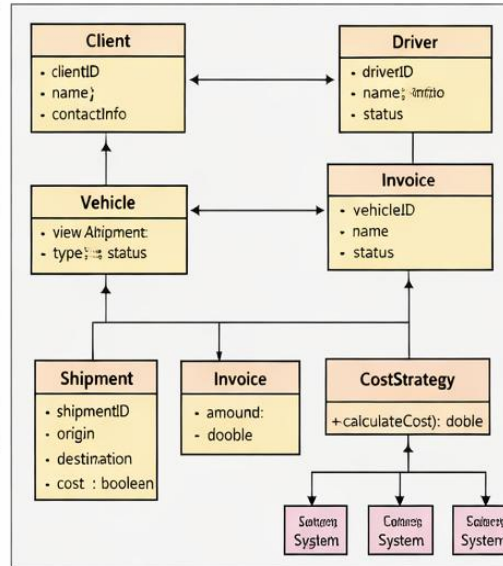
UML Diagrams for SLTMS

Case Study: Impano Gateway Logistics Ltd

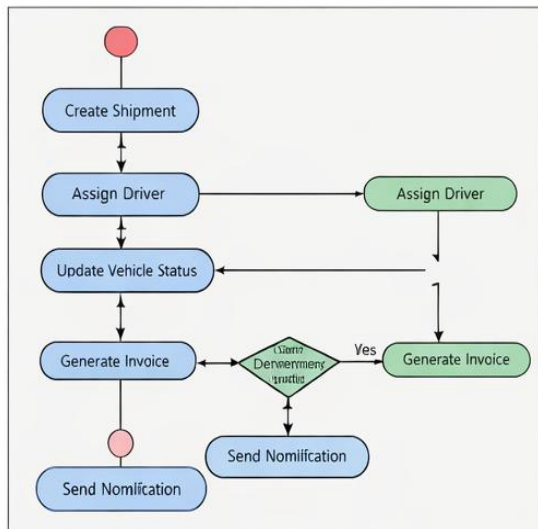
Use Case Diagram



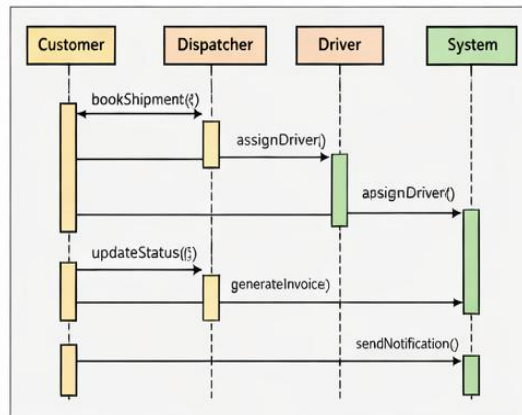
Class Diagram



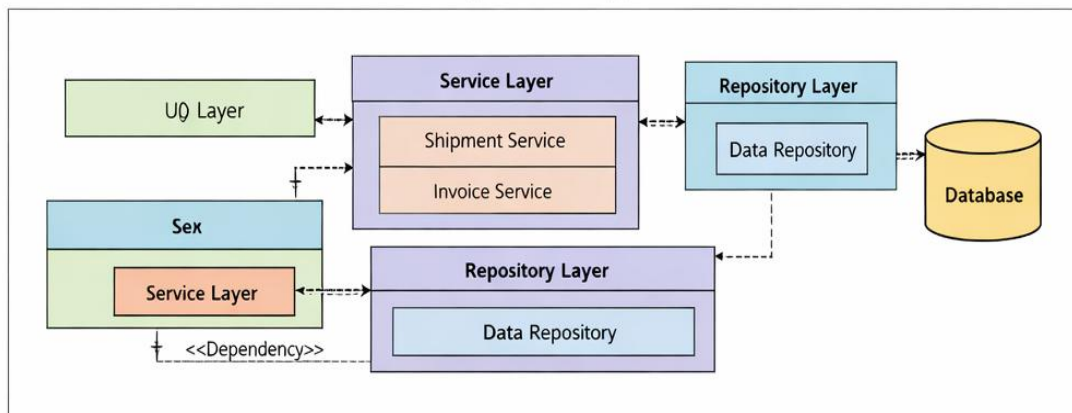
Activity Diagram



Sequence Diagram



Component Diagram



- Use Case Diagram – shows system users and functions
- Class Diagram – shows classes and relationships
- Activity Diagram – explains shipment workflow
- Sequence Diagram – explains object interactions
- Component Diagram – shows system architecture

These diagrams were used to model and analyze the system before implementation.

9. System Architecture

The project uses a layered architecture:

1. Model Layer

Contains domain entities such as Client, Driver, Vehicle, Shipment, and Invoice.

2. Repository Layer

Handles data storage and retrieval.

3. Service Layer

Contains business logic and operations.

4. UI Layer

Provides interaction with users through a console interface.

This architecture improves maintainability and separation of concerns.

10. Design Patterns

Two major design patterns were implemented.

Strategy Pattern:

Used for different shipment pricing formulas depending on vehicle type.

Factory Pattern:

Used for centralized object creation and strategy selection.

Benefits:

- Better code organization

- Improved scalability
- Reduced duplication
- Easier maintenance

11. Dockerization

Docker was used to package the application into a portable container.

Benefits of Docker:

- Consistent environment
- Easier deployment
- Lightweight application packaging
- Improved portability

The project uses a two-stage Docker build:

- Builder stage using Maven
- Runtime stage using Java JRE

12. Version Control with Git

Git was used for version control and project management.

Git features used:

- Repository initialization
- Commit history tracking
- Branch management
- Feature branching
- Remote repository integration

Git improves collaboration and project tracking.

13. Software Testing

The project contains a complete software testing plan.

Testing performed:

- Unit testing
- Functional testing
- Integration testing
- Negative testing

A total of 14 test cases were executed successfully.

Important tested features:

- Shipment creation
- Invoice generation
- Driver assignment
- Cost calculation
- Strategy pattern validation

14. Risks and Limitations

Current limitations include:

- No database persistence
- No authentication system
- Console-only interface
- Single-user operation

Future improvements may include:

- Web interface
- Mobile application
- GPS integration
- Cloud deployment
- Real-time notifications

15. Conclusion

The Smart Logistics & Transportation Management System successfully demonstrates the application of software engineering concepts in solving real-world logistics problems.

The project improved logistics automation through shipment management, fleet coordination, invoice processing, and reporting.

The project also demonstrates:

- Object-oriented programming
- UML modeling
- Design patterns
- Docker containerization
- Git version control

- Software testing

The SLTMS provides a strong foundation for future enterprise logistics solutions.